Enabling GPU Passthrough for microVMs

Guided Research Presentation

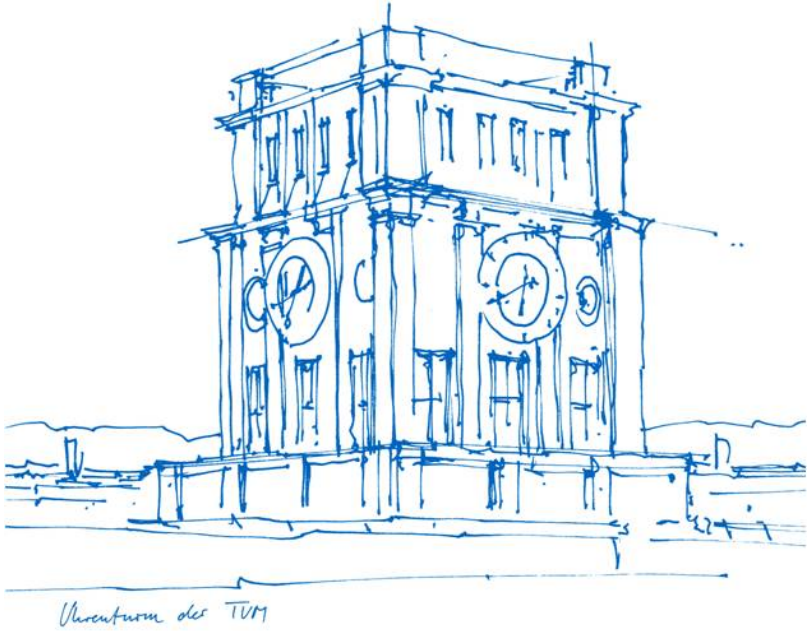
Technical University of Munich
TUM School of Computation, Information and Technology
Chair of Computer Architecture and Parallel Systems

Student: Muhammed Orhun Gale

Advisor: M.Sc. Jianfeng Gu

Examiner: Prof. Dr. Michael Gerndt

Munich, 20 November 2025



Introduction

Motivation: Serverless and microVMs

- **Serverless computing**:
  - Short-lived functions, high elasticity
  - Strong isolation and cost efficiency
- **Requirements**:
  - Very low cold-start latency
  - Small memory footprint per tenant
  - Strong isolation between tenants
- **Firecracker and microVMs**:
  - Minimal device model
  - Designed specifically for serverless workloads

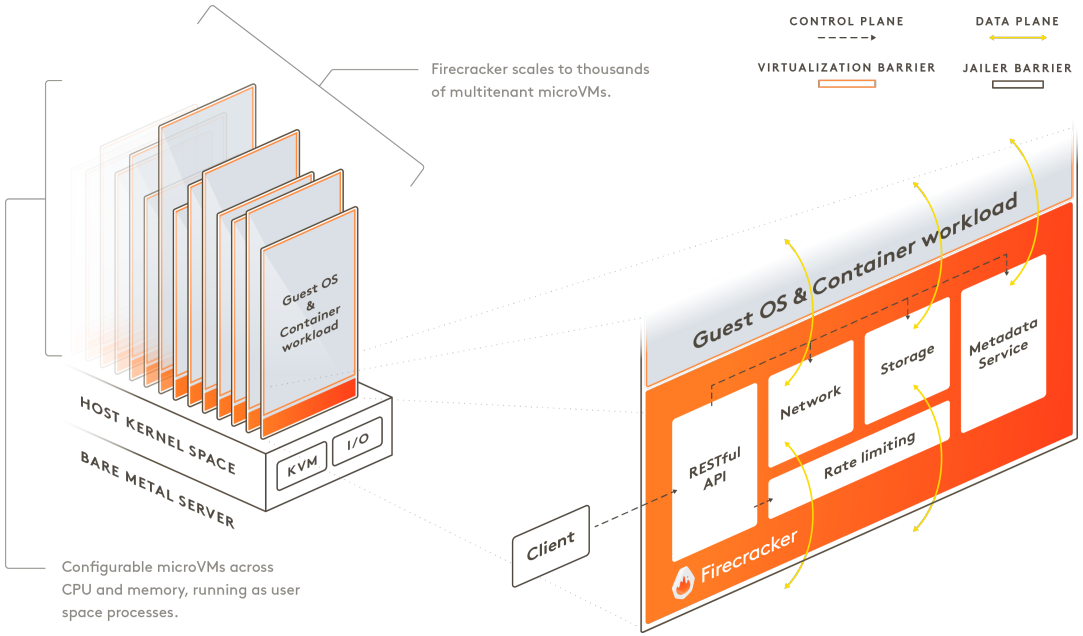


Figure 1: Firecracker microVMs [1]

Introduction

Motivation: GPUs and the Core Question

- **Growing AI/ML workload demand:**
  - Inference and training workloads increasingly dominate cloud usage
  - GPUs are needed for latency and throughput
  - Possible demand for “serverless GPU” capabilities
- **Why not just attach a GPU to a microVM**?

  → Stripped down to be minimal
- **Core question:**
  - How can we use a GPU in a microVM?

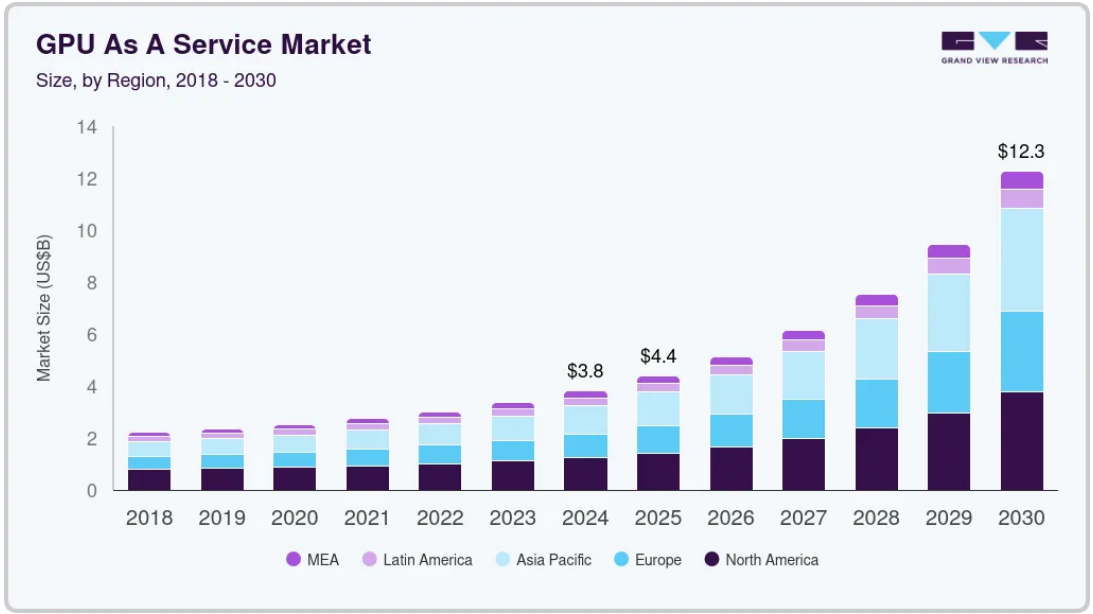


Figure 2: GPUaaS Market Size Forecast [2]

Introduction

Challenges: Why Firecracker Cannot Just Use a GPU

- **Firecracker's minimalism**:
  - No PCI host bridge, no PCI bus → Guest cannot discover a GPU
  - No ACPI MCFG/ECAM → No standard MMIO-based config space access
  - No BAR allocator → Nowhere to map the GPU's MMIO regions
  - No MSI/MSI-X for PCI devices → GPU interrupts cannot reach the guest
  - No VFIO and IOMMU setup → Raw passthrough would let the GPU DMA anywhere

Introduction

Proposed Solution

- **Goal**:
  - Enabling GPU passthrough for Firecracker microVMs
- **Approach**:
  - Add a **minimal PCI stack** to Firecracker
  - Integrate **VFIO** and **iommufd** for safe DMA
  - Implement **MSI/MSI-X** interrupt delivery for PCI devices
  - **Preserve** microVMs **fast boot and small footprint** as far as possible

Introduction

Research Questions

- **Architectural**:
  - What is the minimal PCI functionality required to expose a GPU to a microVM?
  - How can Firecracker safely expose the GPU via VFIO and the IOMMU?
- **Evaluation**:
  - How does passthrough affect cold-start latency, memory footprint and CUDA control-path latency?
  - What happens under GPU oversubscription (multiple microVMs sharing one GPU)?
  - Is this compatible with serverless requirements?

Methodology

Glossary

- **KVM** – Linux hypervisor interface for VMMs to create VMs
- **PCI/PCIe** – Bus for managing hardware devices
- **DMA** – Direct memory access without CPU help
- **VFIO** – Secure device passthrough framework
- **MMIO** – Memory-mapped device registers
- **BAR** – Where a device's MMIO regions are mapped
- **ECAM** – Mechanism to access PCIe configuration space
- **ACPI MCFG** – Table describing where PCIe config space (ECAM) is mapped to
- **eventfd / irqfd** – Event-to-interrupt bridge in KVM

Methodology

How to Passthrough a GPU

- **GPU is a PCIe endpoint:**
  - Exposes config space, BARs, MSI/MSI-X
  - Performs DMA to memory
- **VFIO + IOMMU:**
  - Restrict GPU DMA to guest memory only
  - Protect host and other guests from “unsafe DMAs”
- **Guest**:
  - Sees a normal PCI GPU
  - Runs the unmodified NVIDIA driver and CUDA

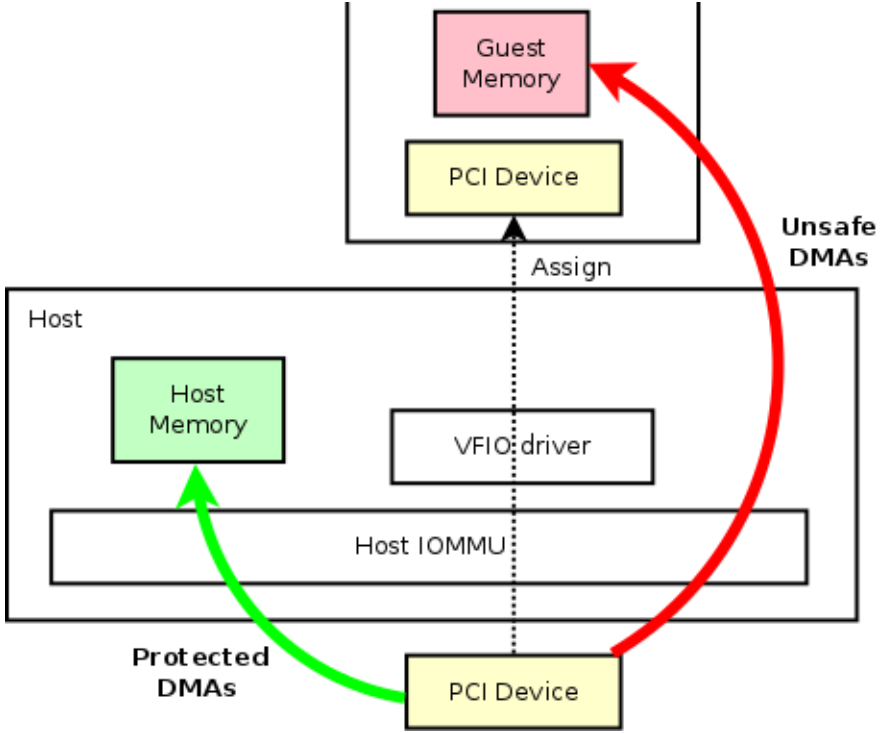


Figure 3: PCI Device Memory Access [3]

Methodology

Architecture Overview

- **Components**:
  - Firecracker (user-space VMM)
  - KVM (vCPUs, guest memory)
  - VFIO core + vfio_pci for the GPU
  - IOMMU + iommufd for DMA isolation
  - microVM with virtio devices and NVIDIA GPU driver

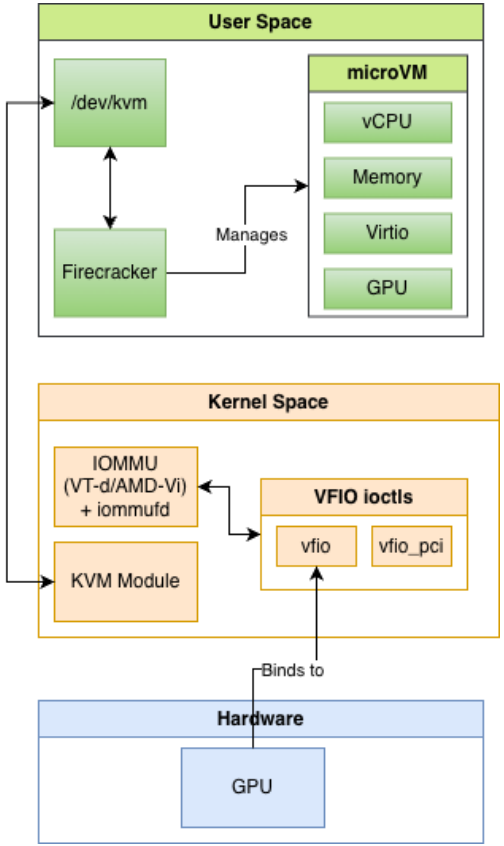


Figure 4: GPU passthrough architecture for Firecracker

Methodology

Architecture Overview

- **GPU lifecycle**:
  - GPU bound to vfio_pci on host
  - Firecracker opens VFIO device and attaches it to a microVM
  - IOMMU ensures GPU can only access microVM memory

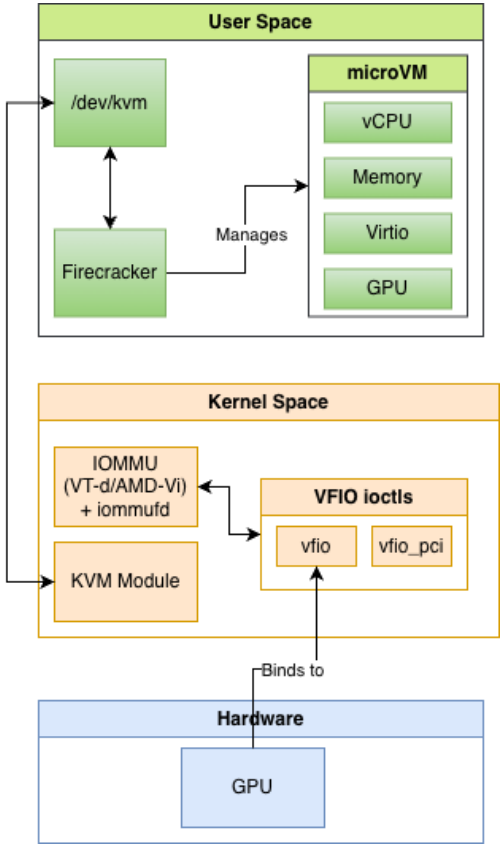


Figure 4: GPU passthrough architecture for Firecracker

Methodology

Implemented Components

- **1. Minimal PCI Stack**
  - Adapted from Cloud Hypervisor (pci crate)
  - PCI host bridge (static)
  - ACPI MCFG for device discovery
  - Config-space interception
  - Guest writes to BARs, MSI, MSI-X
- **2. VFIO Integration**
  - Adapted from Firecracker PCIe PoC branch
  - New VFIO device type to attach host GPU
  - Guest memory mapping into IOMMU DMA domains
- **3. Interrupt Delivery**
  - MSI/MSI-X capability emulation.
  - eventfds allocation for MSI-X vectors
  - VFIO SET_IRQS to route GPU interrupts
  - KVM_IRQFD for fast, direct injection

Methodology

System Flow

- Guest:
  - Enumerates PCI bus via ECAM.
  - Programs BARs and MSI/MSI-X in config space.

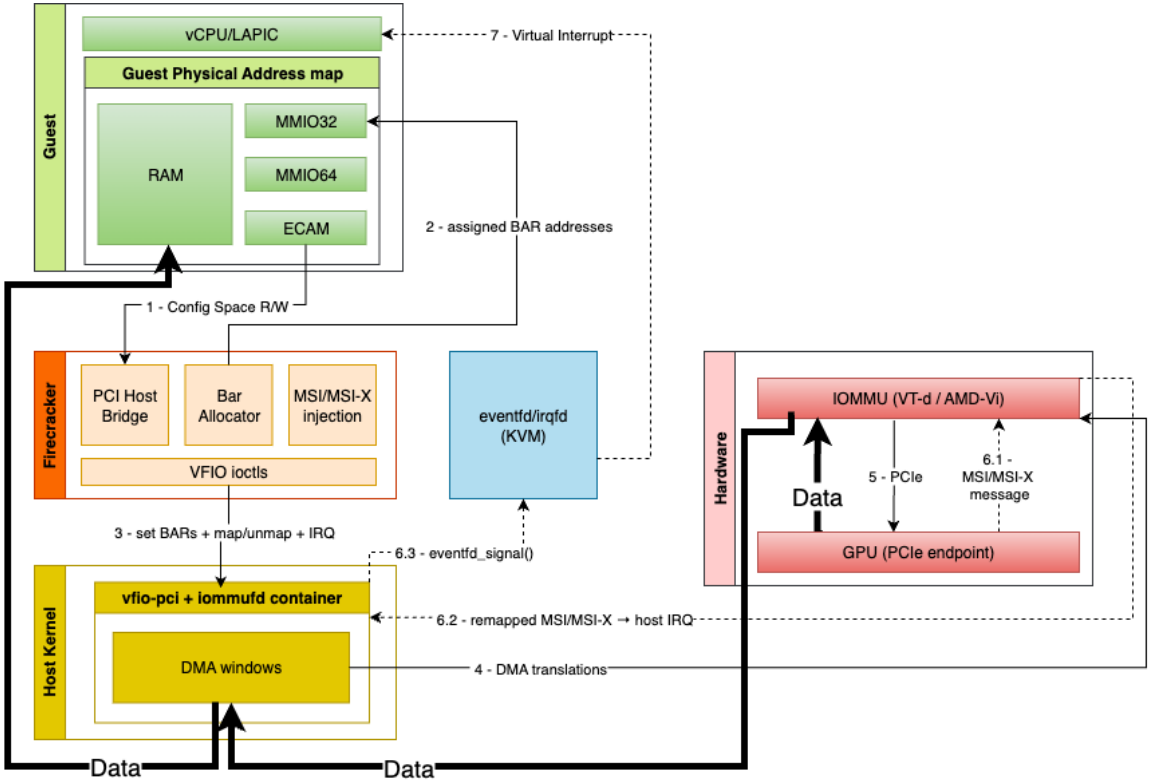


Figure 5: GPU passthrough interaction flow

Methodology

System Flow

- Firecracker:
  - Intercepts config writes.
  - Assigns BAR addresses in MMIO32 / MMIO64.
  - Programs VFIO with BAR mappings and DMA windows.

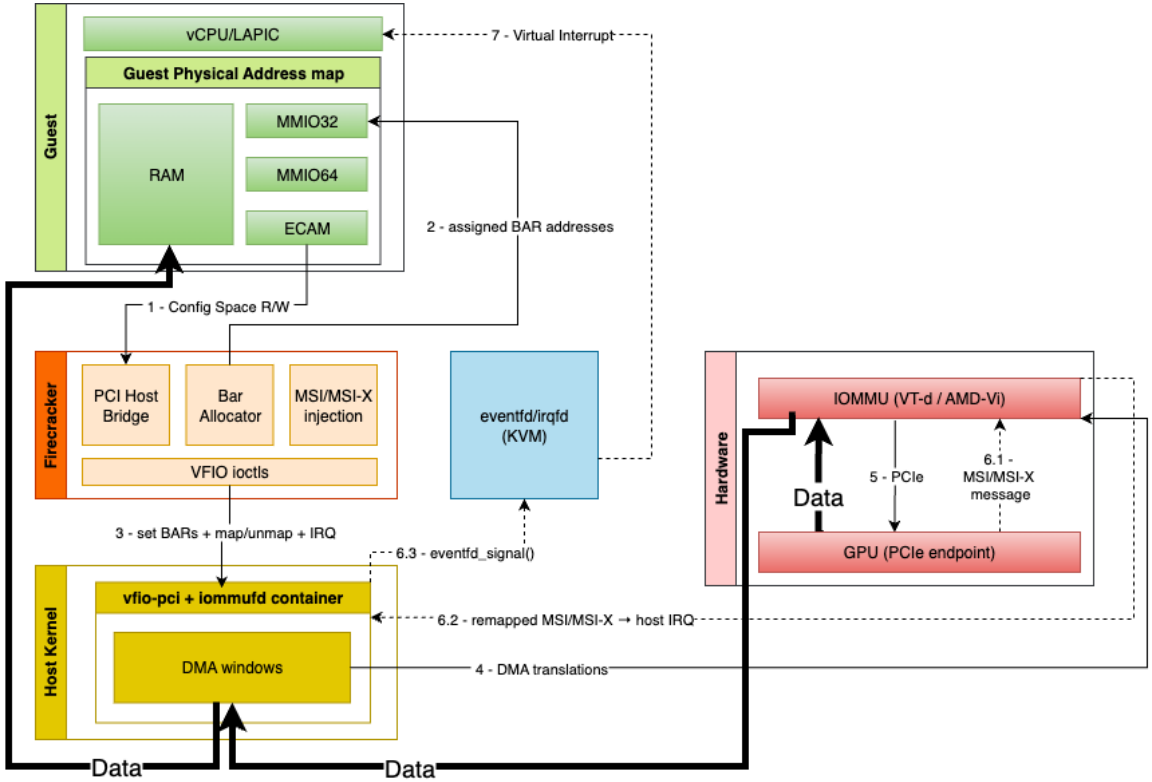


Figure 5: GPU passthrough interaction flow

Methodology

System Flow

- GPU and IOMMU:
  - GPU performs DMA to memory mapped by IOMMU.
  - IOMMU restricts GPU to guest pages.

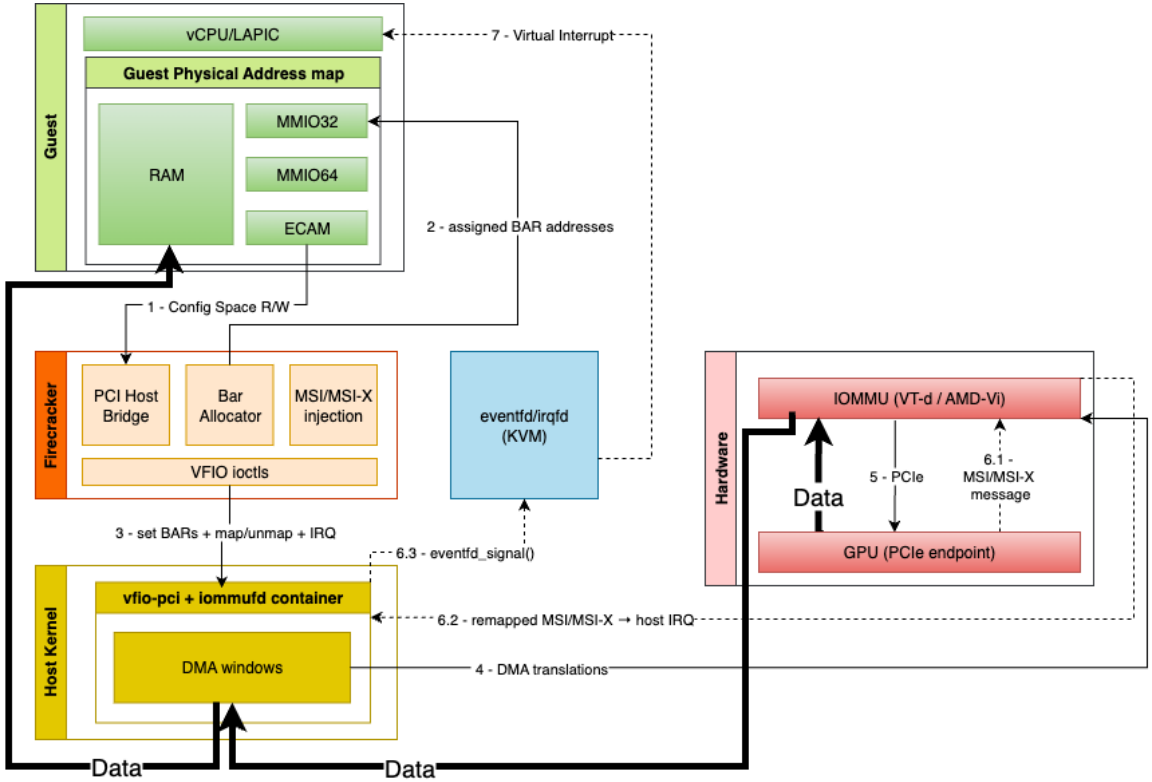


Figure 5: GPU passthrough interaction flow

Methodology

System Flow

- Interrupt path:
  - GPU raises MSI/MSI-X → VFIO → eventfd → irqfd → KVM → guest LAPIC.

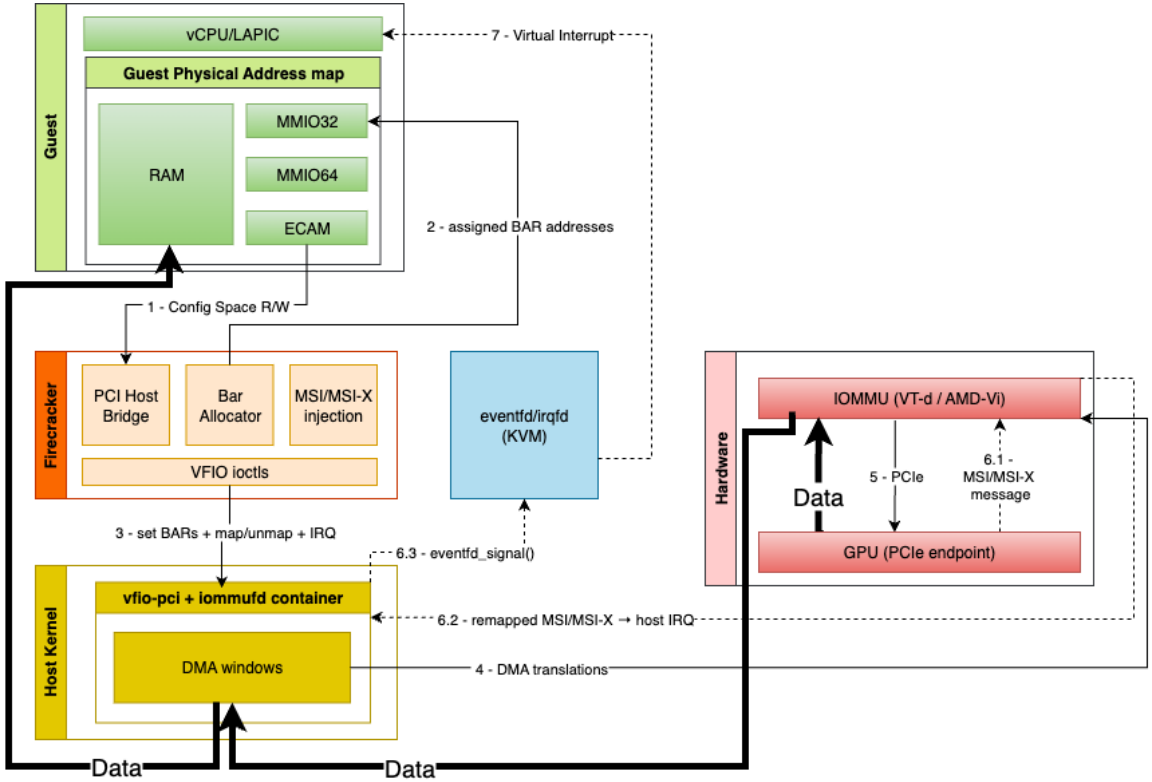


Figure 5: GPU passthrough interaction flow

Experimental Analysis

System Configuration

Component	Configuration
Platform	Dell PowerEdge R7625
CPU	AMD EPYC 9374F (32 cores, x86_64)
Host memory	377.42 GB RAM
GPU	NVIDIA GH100 (H100L, 94 GB)
Host OS	Ubuntu 24.04.2 LTS
Host kernel	Linux 6.8.0-79-generic
VMM	Firecracker v1.11.0 (+ PCI/VFIO extensions)
Guest OS	Minimal Ubuntu 24.04 rootfs
Guest kernel	Generic kernel with PCI, ECAM, MSI/MSI-X, VFIO UAPI
CUDA stack	NVIDIA driver + CUDA runtime matching host driver

Figure 6: Experimental system configuration

Experimental Analysis

Evaluation Methodology

- **Test Environments:**
  - Baseline → Bare-metal execution
  - Experimental Path → microVM + GPU Passthrough
- **Evaluation Metrics:**
  - Boot Time → from VMM launch until GPU is usable in the guest
  - Control-Path Performance → VectorAdd (simple CUDA operation)
  - Compute Performance → DGEMM (heavy matrix multiply)
  - Oversubscription → Behavior when multiple microVMs share one GPU

Experimental Analysis

Boot Time

- **Metric:** Time from firecracker start until the guest can issue a successful CUDA call ("GPU READY").
- **Baseline:** Normal microVM (no GPU) ≈ 1.5 s.
- **Passthrough:** microVM + GPU ≈ 5.1 s (≈ 3.5 s extra for driver + CUDA init).
- **Observation:** Cold-start overhead is dominated by GPU driver and CUDA initialization.

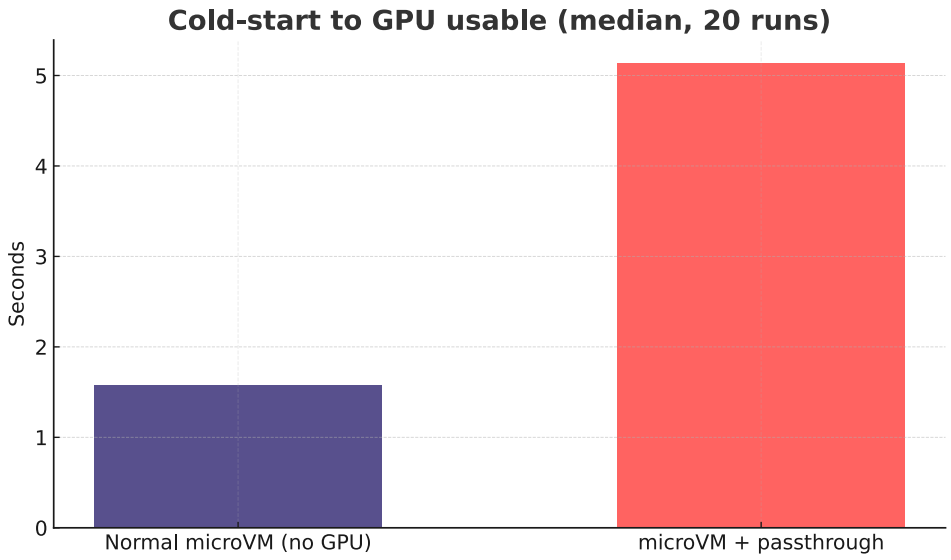


Figure 7: Cold-start time for no-GPU microVM vs. GPU microVM

Experimental Analysis

Control-Path Performance – VectorAdd

- **Metric:** End-to-end latency of a light CUDA workload (context create, transfers, kernel, teardown).
- **Baseline:** Host total time ≈ 190 ms.
- **Passthrough:** microVM + GPU total time ≈ 675 ms.
- **Observation:**
  - Context creation is the main source of slowdown (≈ 5× slower in microVM).
  - H2D/D2H transfers and kernel time are only modestly slower than host.

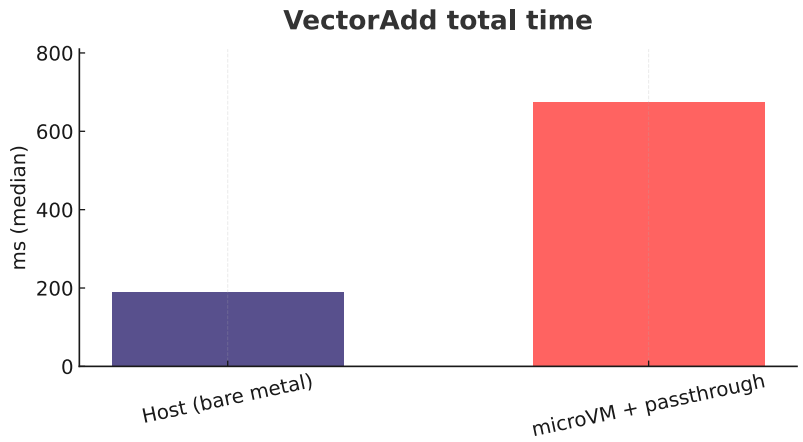


Figure 8: VectorAdd time on host vs. GPU microVM

Host

Ctx	Module	3×Alloc	H2D(A+B)	Host r/t	D2H(C)	Total
113.025	1.55228	0.19952	0.372178	0.02785	0.20797	189.911

microVM with GPU passthrough

Ctx	Module	3×Alloc	H2D(A+B)	Host r/t	D2H(C)	Total
582.257	0.17446	0.748832	0.588542	0.07935	0.29817	674.875

Figure 9: VectorAdd time breakdown

Experimental Analysis

Compute Performance – DGEMM

- **Metric:** DGEMM wall time and GFLOP/s for large matrices.
- **Baseline:** Host – Init ≈ 6.3 ms, DGEMM wall ≈ 1991 ms, ≈ 35.35 TFLOP/s.
- **Passthrough:** microVM – Init ≈ 347 ms, DGEMM wall ≈ 2030 ms, ≈ 34.67 TFLOP/s.
- **Observation:** Init phase is much slower in the microVM, but sustained DGEMM compute is close to bare metal (small loss in GFLOP/s).

Environment	Init (ms)	DGEMM GPU (ms)	DGEMM wall (ms)	GFLOP/s
Host (bare metal)	6.33744	1990.52	1990.58	35351.9
microVM + GPU passthrough	347.352	2029.43	2029.53	34674.1

Figure 10: DGEMM execution time for on host vs. GPU microVM

Experimental Analysis

Oversubscription and Pinned Memory

- **Metric:** Effect of a passthrough microVM and additional guests on host MemAvailable and effective RAM usage.
- **Baseline:** One passthrough microVM with 32 GiB guest RAM on a 386 GiB host.
- **Oversubscribed:** Then boot 8 normal microVMs, each configured with 32 768 MiB RAM (~34.35 GiB).
- **Observations:**
  - Booting the passthrough VM immediately pins ≈34.5 GiB of host RAM for GPU DMA (VFIO + IOMMU, unevictable).
  - Each normal microVM then consumes only ~1–2 MiB of MemAvailable (their RAM stays pageable/overcommitted).
- **Disclaimer:** This large pinned region is specific to our full-device VFIO passthrough; normal systems pin only active GPU buffers, and avoiding it needs designs that map only selected buffers or use vGPU passthrough!

Discussion

- **Impact on serverless**:
  - ~5s GPU cold start is too high for classic “scale to zero”
  - Practical only if microVMs are kept warm with pre-initialized GPUs
- **Performance profile**:
  - Control-path-heavy workloads suffer (context creation, small kernels)
  - Compute-heavy workloads (DGEMM) are close to bare metal after init
  - Data transfer overhead moderate, not dominant

Discussion

Oversubscription and Pinned Memory

- **Oversubscription**:
  - Strong isolation for CPU/memory, but **no isolation** on GPU compute
  - Shared GPU behaves like bare metal under contention: unpredictable latency
- **Role of passthrough**:
  - Suitable baseline and for single-tenant or long-running GPU microservices
  - Insufficient for multi-tenant, highly oversubscribed serverless GPU platform
  - PCI/VFIO plumbing could also bind microVMs to vGPU/MIG slices (if exposed via VFIO), but a separate vGPU/MIG management layer is still required.

Conclusion

- Implemented minimal PCI stack, VFIO integration, MSI/MSI-X delivery, and DMA mapping in Firecracker.
- Enabled unmodified NVIDIA drivers and CUDA workloads inside microVMs via GPU passthrough
- Achieved close-to-bare-metal compute performance after initialization, at the cost of higher cold-start latency and poor GPU sharing behavior

Future Work

- Pre-warmed microVM pools with already initialized GPU contexts
- vGPU / mediated device support (SR-IOV, vendor vGPU) to share GPUs predictably across microVMs
- Integration with GPU partitioning approaches (e.g., MIG) as studied in Ege's thesis

References

- Grand View Research. (2025). GPU as a service market size, share & trends analysis report by component (solution, services), by pricing model (pay-per-use, subscription-based plans), by organization size, by industry vertical, by region, and segment forecasts, 2025–2030. Retrieved from https://www.grandviewresearch.com/industry-analysis/gpu-as-a-service-gpuaas-market-report

- Firecracker Project (2025), Firecracker: Secure and fast microvms for serverless computing, https://firecracker-microvm.github.io

- QEMU Project, Vfio device assignment—common (diagram), https://wiki.qemu.org/images/c/cc/Vfio-device-assignment-common.png, PNG image.

Thank You for Listening!

Any Questions?

Quick Summary:

- **Task:** Enabling GPU Passthough for microVMs
- **Method:** Modifiying and implementing different system components for Firecracker
- **Results:**
  - High cold-start time and poor GPU sharing behavior
  - Close-to-bare-metal compute performance